

プログラム解説

1：制作期間

約6ヶ月（2025年9月上旬～2026年6月上旬）

2：プログラムを組む上で工夫したこと・注意したこと

○キャラクターの状態管理とアニメーション管理

今回制作したゲームは、プレイヤーやボスの行動がアニメーションと密接に連動するため、制作初期に状態管理とアニメーション管理の仕組みを構築しました。

状態管理には有限状態機械（FSM）を採用し、待機・移動・ジャンプ・攻撃・ガードなどの行動を状態ごとに分離しています。

各状態の処理を関数化することで、switch 文の肥大化を防ぎ、処理の責務を明確化しました。

また、状態遷移に応じてアニメーション再生や入力受付を制御することで、行動とアニメーションを一元的に管理しています。

これにより新規アクションの追加や調整が容易となり、開発効率と保守性の向上を実現しました。

このシステムは現在参加している 3D ゲームのチーム制作にも導入しており、メンバーからも扱いやすく拡張しやすい仕組みとして評価をいただいています。

○Effekseer の管理システム

今回制作したゲームは、Effekseer を用いてエフェクトを実装していますが、配布されたライブラリをそのまま使用すると、エフェクトのロード処理や管理コードが各所に分散し、管理対象の把握が難しいという課題がありました。

そこで Effekseer をラップした独自の管理システムを作成し、エフェクトを enum で一元管理する仕組みを構築しました。

これにより、エフェクト名から直接アクセスできるようになり、配列番号を意識する必要がなくなりました。

結果としてコード量の削減だけでなく、エフェクト管理の可読性・保守性が向上し、新規エフェクトの追加や修正も容易になりました。

3：苦勞した所

○足場とプレイヤーの当たり判定

ステージ上の足場とプレイヤーの当たり判定は、DxLib の MV1CollCheck を使用せず、自力計算によって実装しました。

本作では多数の足場オブジェクトを配置することを想定していたため、処理負荷の軽減を目的として独自実装を選択しました。また、衝突時にはプレイヤーが足場のどの面に接触したのかを判定し、最適な押し戻し方向を算出することで、自然な挙動を実現しています。

特に動く足場との当たり判定には苦勞しました。当初は足場との押し戻し処理を行った後、足場の移動量をプレイヤーへ反映する方式を採用していました。

しかし、処理順序の問題により、プレイヤーが足場から浮いたり、逆にめり込んだりする不具合が発生しました。

原因を調査した結果、押し戻し処理と足場移動量の反映タイミングが競合していることが判明しました。

そこで、プレイヤー側で接触した足場の情報を保持し、足場側から移動情報を受け取る構造へ変更することで処理順序を整理しました。

その結果、動く足場上でも安定した当たり判定と移動処理を実現し、不具合の解消に成功しました。

○プレイヤーカメラとターゲットカメラの設計・改良

当初はプレイヤーを中心に追従するプレイヤーカメラを実装し、その後、敵を注視するターゲットカメラを追加しました。

しかし、カメラを切り替えた際に座標が瞬時に変更されるため、視点が急激に移動し、違和感のあるゲーム体験となっていました。

そこで、実際のカメラ座標とは別に目標座標を用意し、補間処理によって現在位置から徐々に移動させる仕組みを実装しました。

これにより、頻繁に発生するカメラ切り替えを自然に行えるようになり、操作時の快適性を向上させることができました。

また、当初のプレイヤーカメラは Y 軸回転のみを想定して実装していたため、新たに制作したプラットフォームアクションでは必要となる X 軸回転への対応が困難でした。

そこで、外積による座標計算から行列計算を用いた方式へ変更し、X 軸・Y 軸回転と座標計算を統一的に処理するよう改良しました。

しかし、この修正をプレイヤーカメラのみに適用した結果、ターゲットカメラとの計算方式に差異が生じ、カメラ切り替え時に不整合によるバグが発生しました。

最終的にはターゲットカメラ側も同様の計算方式へ統一することで問題を解決しました。

この経験から、機能追加を見据えた設計の重要性と、システム全体の整合性を意識して改修を行うことの大切さを学びました。